

Wisdom & Chance TCG — Mobile Payment Integration Sync Guide

Date: March 29, 2026 | Server Version: 2.5.0 (Post-Task #8)

Document Purpose

This document provides complete instructions for an AI agent building the Wisdom & Chance TCG mobile app to integrate the payment system (PayPal, Stripe/Google Pay, and in-game currency purchases) that was added in server Task #8. The mobile app should consume the existing backend payment endpoints as-is; no server changes are needed. This guide supplements the original Mobile App AI Agent Sync Guide (v2.4.0) with all new payment endpoints, data models, product catalog, and implementation flows.

Table of Contents

- 1. Payment System Overview
- 2. Payment Configuration Endpoint
- 3. Product Catalog & Data Model
- 4. Complete Payment API Reference (12 endpoints)
- 5. Purchase Flows (Step-by-Step)
- 6. Battle Pass Premium Track
- 7. Economy Constants & Pricing
- 8. Error Handling Reference
- 9. Mobile Implementation Checklist
- 10. Critical Implementation Notes
- 11. Updated Server File Reference

1. Payment System Overview

The server now supports four payment methods:

Method	Description	Use Case
Gold	In-game currency earned from matches	Packs, bundles, battle pass
Gems	In-game currency purchased with real mon	Packs, bundles, battle pass
Stripe	Credit/debit card + Google Pay + Apple P	Real-money purchases (gem packs, bundles
PayPal	PayPal checkout	Real-money purchases (gem packs, bundles

Key Rules:

- All USD prices are whole dollars (no cents)
- 100 gold = \$1 USD; 100 gems = \$1 USD
- Gem packs (gems_100, gems_550, etc.) are real-money ONLY — isCurrencyPurchasable: false
- All shop packs support BOTH gold AND gems as payment
- Payments route to PayPal account: reagle2808@aol.com
- One-time purchases are enforced server-side with transactional locking

2. Payment Configuration Endpoint

Before showing any payment UI, fetch the payment configuration to know which providers are available.

Endpoint: GET /api/payments/config

Authentication: None required

Response:

```
{
  "paypalClientId": "AaBbCc123...",
  "stripePublishableKey": "pk_live...",
  "paypalEnabled": true,
  "stripeEnabled": true
}
```

Mobile Implementation:

- Fetch on app startup alongside feature flags (GET /api/config)
- Only show Stripe button if stripeEnabled === true
- Only show PayPal button if paypalEnabled === true
- Cache result; re-fetch on app foreground resume
- If both are disabled, only show gold/gems purchase options

3. Product Catalog & Data Model

3.1 PurchaseProduct Schema

```
interface PurchaseProduct {
  id: string;           // e.g. "gems_100", "premium_battle_pass"
  name: string;         // Display name
  description: string;  // Display description
  productType: string; // "gems" | "pack_bundle" | "battle_pass" | "bundle"
  priceUsd: number;    // USD price (whole dollars)
  priceGold: number;   // Gold price (0 = not purchasable with gold)
  priceGems: number;   // Gems price (0 = not purchasable with gems)
  gemsAmount: number;  // Gems granted on purchase (for gem packs)
  packsJson: string;   // JSON array of {type, count} for pack bundles
  dustAmount: number;  // Dust granted on purchase
  isOneTimePurchase: boolean; // Can only buy once per account
  isCurrencyPurchasable: boolean; // Can buy with gold/gems (false = real money only)
  isActive: boolean;   // Currently available
  badgeText: string | null; // UI badge ("Most Popular", "Best Value", etc.)
  sortOrder: number;   // Display order
  createdAt: string;   // ISO timestamp
}
```

3.2 Complete Product Catalog (Seeded on Server)

Gem Packs (Real Money Only)

ID	Name	USD	Gems Granted	Badge
gems_100	Starter Gems	\$1	100	—
gems_550	Popular Gems	\$4	550 (50 bonus)	Most Popular
gems_1400	Value Gems	\$8	1,400 (200 bonus)	Great Value
gems_3600	Mega Gems	\$18	3,600 (600 bonus)	—

gems_8000Ultimate Gems\$358,000 (1,500 bonus)Best Value

All gem packs: isCurrencyPurchasable: false — cannot buy gems with gold/gems.

Pack Bundles (Gold, Gems, or Real Money)

ID	Name	USD	Gold	Gems	Contents
premium_pack_bundle	Premium Pack Bundle	\$3	300	300	3x Premium packs
element_mega_pack	Element Mega Pack	\$6	600	600	5x Element packs (1 each)
legendary_hunt	Legendary Hunt Pack	\$8	800	800	5x Premium packs (boosted Legendary)

Battle Pass

ID	Name	USD	Gold	Gems	Effect
premium_battle_pass	Premium Battle Pass	\$5	500	500	Unlocks premium reward track

Bundles

ID	Name	USD	Gold	Gems	Contents	One-Time?
starter_bundle	Starter Bundle	\$10	1,000	1,000	1,500 gems + 5 premium packs + 500 dust	Yes
season_pass_bundle	Season Pass Bundle	\$15	1,500	1,500	Premium Battle Pass + 10,000 gems + 10 pr	No

3.3 PurchaseTransaction Schema

```
interface PurchaseTransaction {
  id: string;           // UUID
  userId: string;
  productId: string;    // References PurchaseProduct.id
  paymentMethod: string; // "gold" | "gems" | "stripe" | "paypal"
  paymentId: string | null; // Stripe session ID or PayPal order ID
  amountUsd: number;    // USD amount charged (0 for currency purchases)
  currencySpent: number; // Gold/gems spent (0 for real-money purchases)
  status: string;       // "pending" | "completed" | "failed"
  createdAt: string;    // ISO timestamp
}
```

4. Complete Payment API Reference

All payment endpoints use Bearer token authentication: Authorization: Bearer <jwt_token>

4.1 Product & Config Endpoints

GET /api/payments/products

List all active products in the catalog.

Response: Array of PurchaseProduct objects, sorted by sortOrder.

GET /api/payments/config

Get payment provider availability (see Section 2).

GET /api/payments/history

Get the authenticated user's purchase transaction history.

Response: Array of PurchaseTransaction objects, sorted by createdAt descending.

4.2 Purchase Status Checks

POST /api/payments/check-purchased

Check if a user has already purchased specific one-time products.

Single product check:

```
// Request
{ "productId": "starter_bundle" }

// Response
{ "purchased": true }
```

Batch check (multiple products):

```
// Request
{ "productIds": ["starter_bundle", "season_pass_bundle"] }

// Response
{ "starter_bundle": true, "season_pass_bundle": false }
```

4.3 Unified Purchase Endpoint

POST /api/payments/purchase

Routes to the correct payment handler based on paymentMethod.

Request:

```
{
  "productId": "gems_550",
  "paymentMethod": "stripe"    // "gold" | "gems" | "stripe" | "paypal"
}
```

Response varies by payment method:

For gold/gems — immediate fulfillment:

```
{
  "message": "Purchase successful",
  "transaction": { ... },
  "cards": [ ... ],    // Array of pulled cards (for pack products)
  "currencies": { "gold": 450, "gems": 100, "dust": 0 }
}
```

For stripe — redirect to checkout:

```
{
  "action": "redirect",
  "url": "https://checkout.stripe.com/...",
  "sessionId": "cs_..."
}
```

For paypal — redirect to approval:

```
{
  "action": "paypal_approve",
  "orderId": "ORDER_ID",
  "approveUrl": "https://www.paypal.com/checkouts/..."
}
```

Rate limited: 10 requests per 60 seconds per user.

4.4 Currency Purchase (Direct)

POST /api/payments/purchase-currency

Purchase a product using gold or gems directly.

Request:

```
{
  "productId": "premium_pack_bundle",
  "currencyType": "gold"    // "gold" | "gems"
}
```

Success Response:

```
{
  "message": "Purchase successful",
  "transaction": { "id": "...", "productId": "premium_pack_bundle", ... },
  "cards": [
    { "cardId": "fire_001", "rarity": "Rare", "isNew": true, "cardName": "Flame Serpent", "element": "fire", "
  ],
  "currencies": { "gold": 200, "gems": 100, "dust": 0 }
}
```

Error Responses:

- 400 — Product not found, insufficient balance, not purchasable with currency
- 400 — "This one-time purchase has already been claimed"

4.5 Stripe Endpoints

POST /api/payments/stripe/create-checkout

Create a Stripe Checkout session for a product.

Request:

```
{ "productId": "gems_550" }
```

Response:

```
{
  "sessionId": "cs_test_...",
  "url": "https://checkout.stripe.com/c/pay/cs_test_..."
}
```

Mobile Flow: Open url in an in-app browser/WebView. After payment, Stripe redirects to a success URL containing session_id. Extract it and call verify-session.

GET /api/payments/stripe/verify-session?session_id=cs_test_...

Verify a completed Stripe session and fulfill the purchase.

Response:

```
{
  "message": "Purchase successful",
  "transaction": { ... },
  "cards": [ ... ]
}
```

Or if already fulfilled:

```
{
  "message": "Purchase already completed",
  "alreadyFulfilled": true
}
```

GET /api/payments/stripe/success?session_id=cs_test_...

Alias for verify-session. Identical behavior and response.

4.6 PayPal Endpoints

POST /api/payments/paypal/create-order

Create a PayPal order for a product.

Request:

```
{ "productId": "gems_550" }
```

Response:

```
{
  "orderId": "ORDER_ID_123",
  "approveUrl": "https://www.sandbox.paypal.com/checkouts/..."
}
```

Mobile Flow: Open approveUrl in an in-app browser. After user approves payment, extract the orderId from the return URL and call capture-order.

POST /api/payments/paypal/capture-order

Capture (finalize) a PayPal order after user approval.

Request:

```
{
  "orderId": "ORDER_ID_123",
  "productId": "gems_550"
}
```

Response:

```
{
  "message": "Purchase successful",
  "transaction": { ... },
  "cards": [ ... ]
}
```

Server-side validations on capture:

- Verifies status === "COMPLETED"
- Validates custom_id metadata matches userId and productId
- Checks captured amount >= product price
- Idempotent — duplicate captures return existing transaction

4.7 Stripe Webhook (Server-Only)

POST /api/payments/stripe/webhook — Server receives Stripe webhook events. This is NOT called by the mobile app. The server uses it as a safety net for fulfillment if verify-session is never called.

5. Purchase Flows (Step-by-Step)

5.1 Gold/Gems Purchase Flow

```

1. Mobile: GET /api/payments/products !' display catalog
2. Mobile: GET /api/currencies !' check user balances
3. User taps "Buy with Gold" or "Buy with Gems"
4. Mobile: POST /api/payments/purchase-currency
   { productId: "premium_pack_bundle", currencyType: "gold" }
5. Server: Validates balance, deducts currency, grants items atomically
6. Response: { transaction, cards, currencies }
7. Mobile: Show pack opening animation with cards array
8. Mobile: Update local currency display from currencies object

```

5.2 Stripe Purchase Flow

```

1. Mobile: GET /api/payments/config !' confirm stripeEnabled
2. Mobile: GET /api/payments/products !' display catalog
3. User taps "Buy with Card"
4. Mobile: POST /api/payments/stripe/create-checkout
   { productId: "gems_550" }
5. Response: { sessionId, url }
6. Mobile: Open url in in-app browser (SafariVC / Chrome Custom Tab)
7. User completes payment on Stripe checkout page
8. Stripe redirects to success URL with session_id parameter
9. Mobile: Intercept redirect, extract session_id
10. Mobile: GET /api/payments/stripe/verify-session?session_id=<id>
11. Response: { transaction, cards }
12. Mobile: Show success + pack opening if applicable

```

Deep Link / URL Scheme:

The Stripe success_url includes the session_id. Configure your app to intercept:

https://wisdom-and-chance.replit.app/shop?payment=success&session_id={CHECKOUT_SESSION_ID}&product_id={PRODUCT_ID}

For mobile, you may want the server to redirect to a custom URL scheme. Currently the success URL points to the web app.

Your mobile flow should:

- Detect the redirect in the WebView
- Extract session_id from the URL query parameters
- Close the WebView
- Call /api/payments/stripe/verify-session with the session_id

5.3 PayPal Purchase Flow

```

1. Mobile: GET /api/payments/config !' confirm paypalEnabled
2. Mobile: GET /api/payments/products !' display catalog
3. User taps "Buy with PayPal"
4. Mobile: POST /api/payments/paypal/create-order
   { productId: "gems_550" }
5. Response: { orderId, approveUrl }
6. Mobile: Open approveUrl in in-app browser
7. User approves payment in PayPal
8. PayPal redirects back (user closes PayPal UI)
9. Mobile: POST /api/payments/paypal/capture-order
   { orderId: "ORDER_ID", productId: "gems_550" }
10. Response: { transaction, cards }
11. Mobile: Show success + pack opening if applicable

```

5.4 One-Time Purchase Flow (e.g., Starter Bundle)

```
1. Mobile: POST /api/payments/check-purchased
   { productIds: ["starter_bundle"] }
2. Response: { "starter_bundle": false }
3. If false !' show "Buy" button
4. If true !' show "Already Owned" badge, disable purchase
5. Purchase via any method above
6. Server enforces one-time with transactional SELECT FOR UPDATE lock
```

6. Battle Pass Premium Track

6.1 Overview

The battle pass now has a premium track. Certain levels (milestones and late-game) are marked as premium. Players must purchase the Premium Battle Pass product to claim premium rewards.

6.2 Premium Levels

Premium levels are: 10, 20, 25, 30, 35, 40, 45, 50 (milestone levels and levels ≥ 25 at every 5th interval).

6.3 Updated Battle Pass API Response

Endpoint: GET /api/battlepass

Response (updated fields highlighted):


```

{
  "season": {
    "id": "uuid",
    "name": "Season 1: Dawn of the Elements",
    "endsAt": "2026-04-28T00:00:00.000Z",
    "daysRemaining": 30
  },
  "progress": {
    "currentXp": 1500,
    "currentLevel": 7,
    "xpIntoCurrentLevel": 100,
    "xpForNextLevel": 1600,
    "claimedLevels": [1, 2, 3],
    "premiumUnlocked": false // !• NEW: true if user bought
  },
  "levels": [
    {
      "level": 1,
      "xpRequired": 200,
      "rewardType": "gold",
      "rewardAmount": 50,
      "rewardDescription": "50 Gold",
      "isPremium": false, // !• NEW: true for premium
      "claimed": false,
      "unlocked": true
    },
    {
      "level": 10,
      "xpRequired": 2000,
      "rewardType": "pack",
      "rewardAmount": 1,
      "rewardDescription": "1x Standard Pack",
      "isPremium": true, // !• PREMIUM LEVEL
      "claimed": false,
      "unlocked": false
    }
  ]
}

```

6.4 Claiming Rewards

Endpoint: POST /api/battlepass/claim

Request: { "level": 10 }

Error Responses:

- 400 — "Level not unlocked yet" (insufficient XP)
- 400 — "Reward already claimed"
- 403 — "Premium battle pass required to claim this reward" (level.isPremium && !premiumUnlocked)

6.5 Mobile UI Requirements for Premium Track

```

For each battle pass level:
  if level.isPremium === true:
    Show a "Premium" badge/crown icon on the level card
    if progress.premiumUnlocked === false:
      - Dim/grey out the level
      - Show lock icon instead of claim button
      - Tapping should prompt "Upgrade to Premium Battle Pass"
    if progress.premiumUnlocked === true:
      - Show level normally with premium styling (gold/amber accent)
      - Allow claiming when level.unlocked === true
  else:
    Show as normal free-track level

```

6.6 Purchasing Premium Battle Pass

Product ID: premium_battle_pass

- USD: \$5
- Gold: 500
- Gems: 500

After purchase, GET /api/battlepass will return premiumUnlocked: true in the progress object. All previously-locked premium levels become claimable (if XP-unlocked).

7. Economy Constants & Pricing

Currency Exchange Rate

- 100 gold = \$1 USD
- 100 gems = \$1 USD
- Gold is earned through gameplay (matches, quests)
- Gems are primarily purchased with real money

Existing Pack Types (from original shop)

Pack	Gold Cost	Gems Cost	Cards	Odds
Standard	100	100	5	C:60% R:25% E:10% L:5%
Premium	250	250	5	Guaranteed 1+ Rare or better
Element (each)	150	150	5	Filtered to specific element

Match Rewards

- Win: 30 gold + 100 BP XP
- Loss: 10 gold + 30 BP XP
- Draw: 15 gold

Pack Opening Response (cards array)

When a purchase includes packs, the response cards array contains:

```
[
  {
    "cardId": "fire_001",
    "rarity": "Rare",
    "isNew": true,
    "cardName": "Flame Serpent",
    "element": "fire",
    "power": 5
  }
]
```

Use this array to drive the pack opening animation on mobile. `isNew` indicates whether the player already owned this card — highlight new cards.

8. Error Handling Reference

HTTP Status Codes

Code	Meaning	Action
200	Success	Process response normally
400	Bad request / validation error	Show error message to user
401	Unauthorized	Re-authenticate, refresh token
403	Forbidden (e.g., premium required)	Show upgrade prompt
429	Rate limited	Show "Please wait" message, retry after
500	Server error	Show generic error, allow retry
503	Payment provider not configured	Hide that payment method

Common Error Messages

Error	Cause	Mobile Action
"Product not found"	Invalid productId	Refresh product catalog
"Insufficient gold/gems"	Not enough currency	Show "Not enough [currency]" dialog
"This product can only be purchased with	Gem packs can't use gold/gems	Only show Stripe/PayPal buttons
"This product cannot be purchased with [	Price is 0 for that currency	Show other currency option
"This one-time purchase has already been	Duplicate one-time buy	Show "Already Owned" badge
"Payment not completed"	Stripe session unpaid	User may have cancelled; allow retry
"Payment does not belong to this user"	Session/user mismatch	Security error; log and show generic err
"Payment amount mismatch"	Tampered payment	Security error; log and show generic err
"Payment already fulfilled"	Duplicate fulfillment call	Treat as success (idempotent)
"PayPal is not configured yet"	No PayPal credentials	Hide PayPal option
"Stripe is not configured yet"	No Stripe credentials	Hide Stripe option
"Too many requests. Please try again lat	Rate limited	Show cooldown message
"Premium battle pass required to claim t	Claiming premium BP level	Show premium upgrade prompt

9. Mobile Implementation Checklist

Phase 3 Update: Shop & Economy (NEW — Payment Integration)

This replaces the original Phase 3 from the v2.4.0 guide.

3a. Payment Configuration

- & Fetch GET /api/payments/config on startup
- & Store stripeEnabled, paypalEnabled, stripePublishableKey, paypalClientId
- & Only show enabled payment methods in UI

3b. Premium Store / Product Catalog

- & Fetch GET /api/payments/products to get full catalog
- & Group products by productType in UI:
- "Gem Packs" section for productType: "gems"
- "Pack Bundles" section for productType: "pack_bundle"
- "Battle Pass" section for productType: "battle_pass"
- "Special Bundles" section for productType: "bundle"
- & Show badgeText as a ribbon/badge on product cards
- & Show dual pricing: gold price AND gems price (when isCurrencyPurchasable)
- & Show USD price for all products
- & Disable gold/gems buttons when isCurrencyPurchasable === false

3c. One-Time Purchase Tracking

- & On shop load, call POST /api/payments/check-purchased with all one-time product IDs
- & Show "Already Owned" state for purchased one-time items
- & Disable buy buttons for already-purchased one-time items

3d. In-Game Currency Purchase Flow

- & "Buy with Gold" button !' POST /api/payments/purchase-currency { currencyType: "gold" }
- & "Buy with Gems" button !' POST /api/payments/purchase-currency { currencyType: "gems" }
- & Show confirmation dialog before purchase with cost breakdown
- & Smart default: if user can't afford gold, default to gems (and vice versa)
- & After purchase: update currency display from response currencies object
- & After purchase: trigger pack opening animation if cards array is non-empty

3e. Stripe Purchase Flow

- & "Pay with Card" button !' POST /api/payments/stripe/create-checkout
- & Open returned url in in-app browser (SFSafariViewController / Chrome Custom Tabs)
- & Intercept success redirect URL, extract session_id query parameter
- & Call GET /api/payments/stripe/verify-session?session_id=<id> to fulfill
- & Handle alreadyFulfilled: true as success (idempotent)
- & On cancel redirect, show "Payment cancelled" message

3f. PayPal Purchase Flow

- & "Pay with PayPal" button !' POST /api/payments/paypal/create-order
- & Open returned approveUrl in in-app browser
- & After user returns from PayPal approval:
- Call POST /api/payments/paypal/capture-order { orderId, productId }
- & Handle capture errors (user cancelled, payment failed)

3g. Purchase History

- & Purchase history screen: GET /api/payments/history
- & Show transaction list with: product name, payment method, amount, date
- & Show "No purchases yet" empty state

3h. Existing Shop Endpoints (Still Active)

The original shop endpoints still work and remain unchanged:

- GET /api/shop/catalog — Pack catalog (Standard, Premium, Element)
- GET /api/shop/daily-deals — Daily deal with discount
- POST /api/shop/purchase — Buy pack with gold
- POST /api/shop/purchase-bundle — Buy bundle with gems

These coexist with the new payment endpoints. The new `/api/payments/*` endpoints are the recommended path for the premium store, while the original `/api/shop/*` endpoints handle the base gold shop.

Phase 6 Update: Battle Pass Premium Track

- & Check progress.premiumUnlocked from GET `/api/battlepass`
- & For each level, check level.isPremium
- & Premium-locked levels: show lock icon + "Premium" badge + dimmed styling
- & Premium-unlocked levels: show gold/amber accent + "Premium" badge
- & Tapping locked premium level! show upgrade prompt linking to premium_battle_pass prod
- & After purchasing premium BP! refresh battle pass data to update premiumUnlocked

10. Critical Implementation Notes

Authentication

All `/api/payments/*` endpoints use the same Bearer token authentication as every other endpoint. The unified auth middleware handles both web sessions and mobile JWT tokens identically.

Idempotency

- Stripe verify-session and PayPal capture-order are idempotent
- Calling them multiple times for the same payment returns success without duplicate fulfillment
- The server uses paymentId uniqueness (Stripe session ID / PayPal order ID) to prevent duplicates
- One-time purchases use transactional SELECT FOR UPDATE locks for race condition safety

Rate Limiting

Payment creation endpoints are rate-limited:

- `/api/payments/purchase`: 10 requests per 60 seconds per user
- `/api/payments/purchase-currency`: 10 requests per 60 seconds per user
- `/api/payments/stripe/create-checkout`: 5 requests per 60 seconds per user
- `/api/payments/paypal/create-order`: 5 requests per 60 seconds per user
- `/api/payments/paypal/capture-order`: 5 requests per 60 seconds per user

Mobile should implement client-side debouncing on purchase buttons to avoid hitting rate limits.

Fulfillment Details

When a purchase is fulfilled, the server atomically:

1. Creates a purchase_transaction record with status "completed"
2. Grants gems (if gemsAmount > 0)
3. Grants dust (if dustAmount > 0)
4. Opens packs (if packsJson has entries) — cards are added to collection
5. Unlocks premium battle pass (if product is battle_pass or season_pass_bundle)

All of this happens in a single database transaction. If any step fails, everything rolls back.

WebView Payment Handling (iOS/Android)

iOS (SFSafariViewController):

- Use SFSafariViewController for Stripe/PayPal checkout
- Register a URL scheme or Universal Link for redirect interception
- Dismiss the view controller when redirect is detected

Android (Chrome Custom Tabs):

- Use Chrome Custom Tabs for Stripe/PayPal checkout

- Register an intent filter for redirect URL interception
- The activity stack handles return automatically

Sandbox vs Production

- PayPal currently uses sandbox: <https://api-m.sandbox.paypal.com>
- For production, the server will switch to <https://api-m.paypal.com>
- The mobile app does NOT need to know which environment — the server handles all PayPal API calls
- Stripe environment is determined by the publishable key prefix (pk_test_ vs pk_live_)

11. Updated Server File Reference

New and updated server-side files (mobile app does NOT modify these):

File	Purpose
server/paymentRoutes.ts	All payment REST endpoints (products, pu
server/paymentService.ts	Product catalog, fulfillment logic, curr
server/stripeClient.ts	Stripe client initialization via Replit
shared/models/economy.ts	Purchase products/transactions DB schema
server/routes.ts	Battle pass endpoints updated with isPre

Pre-Existing Files (Unchanged)

File	Purpose
server/economyService.ts	Gold/gems/dust grant helpers, pack openi
server/mobileAuth.ts	Mobile JWT auth (login, refresh, me)
server/unifiedAuth.ts	Unified auth middleware (session + JWT)

Quick Reference Card

PAYMENT CONFIG:	GET	/api/payments/config
PRODUCT CATALOG:	GET	/api/payments/products
PURCHASE HISTORY:	GET	/api/payments/history
CHECK PURCHASED:	POST	/api/payments/check-purchased
BUY WITH CURRENCY:	POST	/api/payments/purchase-currency
UNIFIED PURCHASE:	POST	/api/payments/purchase
STRIPE CHECKOUT:	POST	/api/payments/stripe/create-checkout
STRIPE VERIFY:	GET	/api/payments/stripe/verify-session?session_id=
STRIPE SUCCESS:	GET	/api/payments/stripe/success?session_id=
PAYPAL CREATE:	POST	/api/payments/paypal/create-order
PAYPAL CAPTURE:	POST	/api/payments/paypal/capture-order
BATTLE PASS:	GET	/api/battlepass
CLAIM REWARD:	POST	/api/battlepass/claim
CURRENCIES:	GET	/api/currencies

This document was generated on March 29, 2026 for server version 2.5.0.

For the most current API reference, check GET /api/api-docs or GET /api/admin/sync?code=4838.

The mobile app should be a pure consumer of these APIs. No server modifications required.